

A Universal Positioned-Read Contract for Forensic Evidence: Block-by-Block Filesystem Decoding over Compressed Images, with a Panic-Free Parsing Substrate

Albert Hui
Security Ronin
albert.hui@securityronin.com

Abstract

A digital forensic examiner rarely receives a raw disk. Evidence arrives as an EWF/E01 image, a VMDK or QCOW2 virtual disk, an AFF4 container, a logical AD1 or DAR archive, or a live device — and each carries a filesystem (NTFS, ext4, APFS, HFS+, ex-FAT, ISO 9660, ...) that the examiner must walk to find the artifact that matters. Today every tool re-implements this stack per container×filesystem pair, and each re-implementation re-derives the same two error-prone primitives: how to read a byte range from a decoded image, and how to parse an integer field out of attacker-controllable bytes without crashing.

We describe a small contract that unifies this surface. Every byte-source — however deeply it is compressed, split, or virtualised — exposes a single cursorless *positioned read*, `read_at(&self, offset, buf)`; every filesystem exposes the same inode-addressed tree over that source. We make three observations that follow from this shape. First, the contract is a genuine *universal reader*: eighteen fleet formats (six containers, twelve filesystems and archives) implement one of two traits, and a consumer never branches on input format. Second, because the read primitive is positioned and cursorless, walking a scattered on-disk structure (the NTFS \$MFT, whose records are spread across the volume) over a *block-compressed* container (E01, stored as independently deflated 32 KiB chunks) decompresses only the chunks that overlap each small request — so peak transient decompression memory is approximately one chunk, independent of image size, rather than the whole inflated volume. Third, the parsing substrate beneath all of this is a `no_std`, dependency-free, fuzzed *panic-free bounded reader* whose out-of-range result is 0, never a crash — the correct default for untrusted evidence, and one whose adoption surfaced a real reachable panic (a denial-of-service) in a shipping carver.

We are careful to separate what is *demonstrated* from what is *argued*: the end-to-end NTFS-over-E01 composition is validated for correctness against The Sleuth Kit but exists today as a feature-gated integration proof, not a shipped pipeline; the memory property is an analytical bound over a mechanism we verify in code, not yet a measured resident-set benchmark of an \$MFT walk.

1 Introduction

Forensic acquisition rarely yields a device the analyst can read directly. The evidence is an *image* — a bit-for-bit copy wrapped in a container format that adds compression, checksums, segmentation, or virtualisation. EWF/E01 [1] compresses the media into independently deflated chunks and splits it across .E01, .E02, ... segments; VMDK and QCOW2 store sparse grains and copy-on-write clusters; AFF4 [2] is a Zip-based map of image streams; AD1 and DAR are logical file-tree archives, not disks at all. Inside the decoded media sits a partition table and one or more filesystems, each with its own on-disk layout.

The classical consequence is a combinatorial explosion of glue: a tool that mounts NTFS-inside-E01 shares almost no code with one that mounts ext4-inside-VMDK, even though the only difference is which decoder produces the bytes and which parser consumes them. Each glue path re-derives two low-level primitives, and re-derives them *badly*: (i) how to obtain an arbitrary byte range from the decoded image, and (ii) how to read a length or offset field out of attacker-controllable bytes. The second is a security primitive — an out-of-range read of a corrupt image is a crash (a denial of service) or, worse, a silently wrong value — and it is copied, subtly differently, into every reader.

This paper describes a contract that collapses that

Table 1: Evidence tiers used in this paper.

Tier	Basis
1	Independent third-party oracle (e.g. The Sleuth Kit) or real-world data.
2	Real engine output; ground truth derivable from the construction or verified against the source.
—	Analytical bound over a mechanism verified in code, explicitly not measured.

surface, and two properties that fall out of its shape.

Contributions

1. **A universal reader contract (§2):** two traits — `ImageSource`, a cursorless positioned-read byte source, and `FileSystem`, an inode-addressed tree over it — that eighteen distinct fleet formats implement, so container decode, partition windowing, and filesystem parsing compose without any consumer branching on input format. We report the current, *verified* coverage (§2.1), including what is *not* yet on the contract.
2. **Block-by-block decoding with a one-chunk memory bound (§4):** because the read primitive is positioned, walking the scattered NTFS \$MFT over a block-compressed E01 decompresses only the chunks overlapping each ~ 1 KiB record request into a single reusable scratch page — an analytical bound of $\leq K$ chunks for K records, versus inflating the entire image. We verify the mechanism against the code and validate read correctness against The Sleuth Kit; we are explicit that the resident-memory figure is an analytical bound, not yet a benchmark (§5).
3. **safe-read, a panic-free parsing substrate (§3):** a `no_std`, zero-dependency, fuzzed crate whose bounded integer readers return 0 on any out-of-range offset and never panic. Consolidating ~ 10 hand-rolled copies onto it surfaced a reachable out-of-bounds panic — a denial-of-service on a crafted \$MFT — in a shipping carver, which we fixed under test.

Throughout we label empirical claims by the strength of the check that backs them, following a two-tier scheme (Table 1): *Tier 1* is validated against an independent third-party oracle or real-world data; *Tier 2* is real engine output whose ground truth is derivable from the construction or verified in code. We say “argued” or “analytical” where a claim is a bound over a verified mechanism rather than a measurement.

2 The Universal Reader Contract

The contract has one insight: *everything the examiner opens is a file tree; the only difference is how deep you decode before files appear*. A bare filesystem yields files at the root after one parse; a logical archive (AD1, DAR) is already a file tree; a physical container decodes to a block device whose partition table becomes directories, each subtree a parsed filesystem. Two read-only traits express this.

ImageSource — the byte edge. Any decoded media is a read-only, randomly addressable byte stream. The trait (in `forensic-vfs`) has one required method:

```
pub trait ImageSource: Send + Sync {
    fn len(&self) -> u64;
    fn read_at(&self, offset: u64,
               buf: &mut [u8]) -> VfsResult<usize>;
    // extents(), view(), source_id(): defaults
}
```

There is no cursor and no write method: reads are positioned and parallel-safe by construction (workers share one `Arc<dyn ImageSource>`), and evidence is read-only in the type system — a mutation is uncompileable. A partition is a `SubRange` adapter that adds a base offset and clamps the length, so a read can never escape its window; container decoders (EWF, VMDK, QCOW2, VHDX, VHD, DMG, AFF4) present their decoded output as an `ImageSource`.

FileSystem — the tree edge. A parsed filesystem is an inode-addressed tree, also `&self` over interior mutability so one mounted handle serves N workers. Files are addressed by an opaque `FileId` (a filesystem-specific identity, not a raw integer); directory listings are owned `Send` streams; content is read by the same positioned `read_at(&self, id, stream, off, buf)`. Forensically significant operations — deleted-inode enumeration, unallocated ranges, named data streams (NTFS ADS), hard links, slack — are first-class methods with default-empty implementations, so a reader implements only what its format supports.

Composition. An engine composes these edges: sniff the input, decode a container to an `ImageSource`, detect a volume system and window each partition with a `SubRange`, then parse the filesystem in that window to a `FileSystem`. A logical archive skips the middle and is a `FileSystem` over the file directly. The consumer — a FUSE mount, a timeliner, a batch carver — sees one tree and never learns the input format.

Table 2: Contract coverage, verified by source enumeration. “On contract” = implements the named trait today.

Contract	On contract
<code>ImageSource</code> (containers)	ewf, vmdk, qcow2, vhdx, dmg
<code>FileSystem</code> (filesystems)	ext4, ntfs, apfs, hfs+, fat/exfat,
<code>FileSystem</code> (archives)	zip, dar, ad1
<code>CryptoLayer</code> (FDE)	— (none)
Temporal / shadow copies	—

2.1 Verified coverage, and what is missing

Honesty about coverage matters: the contract’s value is only as real as the fraction of formats that actually implement it. Table 2 reports the status we *verified by enumeration* of the source, not by recollection. Eighteen formats are on the contract; a substantial set is not yet — notably three filesystems (Btrfs, ZFS, ReFS), two container formats (plain VHD, the AFF4 disk side), the entire full-disk-encryption layer (BitLocker, LUKS, FileVault, VeraCrypt — a defined `CryptoLayer` contract with no implementors), and the temporal/shadow-copy dimension, which the fleet models not as a volume but in its own `Snapshot/Store` (raw VSS) and `TemporalCohort/ EpochTag` (historical-state) vocabulary. We report this gap deliberately: a “universal” reader that silently omitted a third of its formats would be a worse claim than an honest partial one.

3 The Parsing Substrate: `safe-read`

Every reader above must extract integers — offsets, lengths, counts — from bytes an adversary may have crafted. The naive idiom, `u32::from_le_bytes([b[o], b[o+1], b[o+2], b[o+3]])`, panics when `o` runs past the buffer, and `o+4` itself can overflow `usize`. A panic on a corrupt image is a denial of service; a tool that instead returns a wrong value is worse. The correct default for untrusted evidence is: an out-of-range read yields 0 and never crashes.

We factor this into `safe-read`, a `no_std`, zero-dependency, `forbid(unsafe)` crate of six bounded readers (`le_u16/u32/u64`, `be_u16/u32/u64`):

```
pub fn le_u32(data: &[u8], off: usize) -> u32 {
    let some(end) = off.checked_add(4) else { return 0 };
    match data.get(off..end) {
        Some(s) => u32::from_le_bytes(s.try_into().unwrap()),
        None => 0,
    }
}
```

The guarantee — “0 out of range, never a panic, for any slice and any offset including `usize::MAX`” — is exactly a property a fuzz target can assert, so the crate ships a `cargo-fuzz` harness that drives arbitrary (slice, offset) pairs and treats any panic as a failure.

The consolidation of `safe-read` is cosmetic. The fleet contained, is 1066 and `bolts` of these readers (five in the NTFS USN parser alone, 70 others in the GPT and live-device layers). Migrating them onto `safe-read` surfaced a real bug: the NTFS \$MFT carver’s local reader indexed directly `entry[attr_offset+20]`, and a crafted record with `attr_offset ∈ [1009, 1015]` drove that index one byte past the 1024-byte record — a reachable out-of-bounds panic, i.e. a denial-of-service on a malicious \$MFT. We reproduced it with a failing test, then fixed it by routing the reads through `safe-read`: the out-of-range read now returns 0, and the record is rejected by the carver’s existing content-length guard rather than crashing the process (Tier 2: a hand-constructed record whose out-of-range behaviour is derivable; the panic and its removal are verified by the test).

4 Block-by-Block Decoding over Compressed Images

The contract’s second payoff is the paper’s central technical claim, and it is a direct consequence of the read primitive being *positioned* rather than a cursor.

The problem. EWF/E01 stores the acquired media as a sequence of *independently compressed* chunks — classically 32 KiB (64 sectors) each, deflated with zlib. Chunk i covers logical bytes $[i \cdot c, (i+1) \cdot c)$ for chunk size c . To read logical byte x one must inflate the chunk $\lfloor x/c \rfloor$; the compressed chunks are not randomly addressable in the plaintext.

Now consider the NTFS \$MFT: a table of fixed 1024-byte records that describe every file, but which is itself an ordinary (often fragmented) file, so its records are scattered across the volume at offsets that must be resolved through the \$MFT’s own data-run list. A timeline or a triage tool reads *many* records at scattered offsets. The naive way to serve those reads from an E01 — decompress the whole image to a temporary file, then seek — costs resident/temporary storage equal to the *full inflated media*, which for a multi-terabyte acquisition is prohibitive.

The mechanism. Because `ImageSource::read_at` is a bounded byte-range request, the E01 reader can serve it by inflating only the chunks that overlap the range. Its `read_at` loops, and per iteration reads within a single chunk:

```

let chunk_id = (off / self.chunk_size) as usize;
let page = self.read_chunk(chunk_id)?; // inflate one
    chunk
let po = (off % self.chunk_size) as usize;
buf[b..b+to_read]
    .copy_from_slice(&page[po..po+to_read]);

```

Decompression allocates exactly one chunk-sized scratch page (`vec![0u8; chunk_size]`) and inflates into it, so *peak transient decompression memory is $\approx$ one chunk, independent of image size*. An NTFS read of a 1024-byte record therefore resolves its physical offset through the run list, issues a positioned read, and lands inside a single 32 KiB chunk (32 records share a chunk; a straddle touches two). A partition offset is added by a `SubRange` on the way down, transparently.

The bound. Reading K \$MFT records touches at most $\sim K$ chunks — at most $K \times 32$ KiB inflated *transiently* — and far fewer in practice, since 32 consecutive records share a chunk and a small LRU chunk cache (the only interior mutation on the read path) keeps recently touched chunks warm, so neighbouring records and re-reads pay no repeat decompression. The steady-state resident footprint is then bounded and *configurable*: `cache_size` chunks (e.g. 100 chunks $\approx$ 3.2 MB, 1000 $\approx$ 32 MB), not the inflated image. The contrast is stark: $O(1)$ transient plus a bounded cache, versus $O(\text{image})$ to reach records scattered across the disk.

Crucially, this is not an E01-specific hack. The *same* property holds for any block/cluster/grain-addressed container — VMDK grains, QCOW2 clusters, VHDX BAT blocks — because each implements the identical positioned-read contract. The optimization is a property of the *contract*, realised once per decoder, not re-coded per consumer.

5 Evaluation

We evaluate three claims at the tiers we can honestly support, and are explicit about what is not yet measured.

The contracts compose and read correctly (Tier 1). An end-to-end integration test opens a real .E01, wraps `EwfReader` as an `ImageSource`, parses NTFS over it, resolves `file1.txt` (\$MFT record 37, 408 bytes), and compares the recovered bytes and metadata against The Sleuth Kit’s `fls/icat` on the same image. The read path therefore flows small positioned reads down to per-chunk inflation, and the recovered content matches an independent oracle. *Caveat*: this composition is gated behind a `vfs` feature with local path dev-dependencies — it is a demonstrated proof, not a shipped product pipeline. The orchestrating binary (`disk-forensic`) today uses EWF only for partition-

layout analysis and does not yet mount NTFS over it end-to-end.

The one-chunk memory bound (analytical; mechanism verified). We verify in code that decompression allocates a single chunk page and that `read_at` inflates only overlapping chunks; the $\leq K \times 32$ KiB transient bound follows analytically. We do *not* yet report a measured resident-set benchmark of an \$MFT walk: the repository benchmarks the *chunk-table* residency (eager vs. lazy population — an eager table is $\approx \text{chunk_count} \times 16$ B, ~ 1 GB for a 2 TB image) but not the peak decompressed RSS of a filesystem walk. We state the memory figure as an analytical bound over a verified mechanism, and flag a purpose-built measurement as the natural next step.

Panic-freedom (property, fuzzed) and a real bug (Tier 2). `safe-read`’s never-panic property is asserted by a fuzz target over arbitrary (slice, offset) inputs. The \$MFT carver panic it surfaced is reproduced by a deterministic failing test and removed by the fix; the crafted record’s out-of-range behaviour is derivable from the NTFS record layout.

Scope. These results establish that the contract *composes correctly* and that the block-by-block property is *mechanically real*; they do not yet establish an end-to-end production pipeline nor a measured memory advantage on a large image. We name both as the outstanding empirical work rather than imply them.

6 Related Work

Container decoding and filesystem parsing are individually mature: `libewf` [1] for EWF, The Sleuth Kit [3] for NTFS/ext/HFS+, and the `libyal` family for many formats. What these provide as separate libraries with bespoke APIs, our contract unifies behind two traits, so *composition* (container $\rightarrow$ volume $\rightarrow$ filesystem) is expressed once. `dfVFS` [4] (Plaso) is the closest prior unification — a Python virtual file-system layer over many back-ends — and validates the goal; our contribution is (i) a positioned, cursorless, read-only, `Send+Sync` byte edge that makes the block-by-block memory property fall out for *any* block-compressed container, and (ii) an explicit, fuzzed panic-free parsing substrate as a first-class dependency rather than an idiom re-copied per reader. The memory argument is an instance of demand-paged decompression; we make it a property of a shared contract rather than a per-tool optimization.

7 Discussion and Limitations

Threats to validity — construct. The “universal” claim is bounded by Table 2: three filesystems, two containers, all of FDE, and the shadow-copy dimension are not yet on the contract. We report coverage as measured, not as aspiration.

Threats to validity — internal. The end-to-end composition is a feature-gated test, and NTFS currently reaches the `ImageSource` through a `Read+Seek` cursor adapter behind a mutex rather than issuing raw parallel `read_at`; this preserves the sliding-window decompression property but serialises reads and does not yet exploit the contract’s inherent concurrency.

Threats to validity — external. The memory bound is analytical; a different container’s chunk/cluster size, or a filesystem with larger scattered reads, changes the constant (“chunks overlapping the range,” not strictly one). The LRU cache trades a bounded, configurable resident footprint for avoided recompression — “ $\approx$ one chunk” is the *transient* bound, not the resident one.

8 Conclusion

A single positioned-read contract turns a combinatorial glue problem into two composable edges, and two useful properties follow from its shape rather than from per-tool effort: scattered filesystem walks over block-compressed images decode one chunk at a time, and a shared fuzzed parser makes “don’t crash on a corrupt image” the default instead of an idiom re-derived — and mis-derived — per reader. We have been deliberate about the boundary between what is demonstrated (correct composition against an independent oracle; a real panic fixed under test) and what is argued (the resident-memory advantage, an analytical bound awaiting a benchmark), and about how much of the format surface is genuinely covered today. The outstanding work is concrete: finish the format coverage, wire the composition into a shipping pipeline over a raw positioned-read source, and measure the \$MFT-walk memory advantage on a multi-terabyte acquisition.

References

- [1] Joachim Metz. libewf: Library and tools to access the expert witness compression format (ewf). <https://github.com/libyal/libewf>, 2006. Accessed 2026.
- [2] Michael Cohen, Simson Garfinkel, and Bradley Schatz. Extending the advanced forensic format to accommodate

multiple data sources, logical evidence, arbitrary information and forensic workflow. In *Proceedings of the Digital Forensic Research Workshop (DFRWS)*, 2009. AFF4.

- [3] Brian Carrier. *File System Forensic Analysis*. Addison-Wesley, 2005. The Sleuth Kit: <https://www.sleuthkit.org/>.
- [4] Log2timeline. dfVFS: Digital forensics virtual file system. <https://github.com/log2timeline/dfvfs>, 2016. Accessed 2026.